# Software Engineering Project

**CampusCare**

**Submitted at the Faculty of Economics and Business Administration**

**German International University**


**Submitted by:**

Gamila Anwar

Habiba Ahmed

Jana Kassem

Malak Madyan

Miriam Ramy


**Tutorial No.:**

3


Dr. Iman Awaad

Mr. Moamen Alaa


Date:

17/05/2026

# 1. Introduction

This project introduces a mobile-based facility management system intended to help with the effective maintenance and preservation of university campus assets. The app offers a digital solution that allows members of the university community to submit infrastructure, cleanliness, sustainability, and maintenance concerns using an efficient and user-friendly mobile app. By digitizing the reporting and handling process, the system aims to enhance communication, shorten response times, and ensure a safe, clean, and well-maintained campus environment.

The system is designed for several university user groups, each with specific responsibilities and permissions. Faculty members and students can report problems and check their status, while facility managers are in charge of analyzing filed complaints, allocating tasks to maintenance staff, and tracking progress. Maintenance staff receive job assignments and offer progress updates after addressing issues. An administrator role manages users, permissions, and general system operations, resulting in a secure and structured workflow.

The software's scope involves the development of a mobile application. Core functions include issue submission with photos, descriptions, and location details, issue categorization, role-based access control, task assignment, commenting, priority management, and real-time status updates via dashboard views.

## 1.1. Product Vision & Scope

The product envisions a digitally united campus in which facility-related issues are identified and resolved with minimum delay and maximum clarity. It aims to improve how campus-related problems are addressed and managed by replacing informal or fragmented systems with a centralized and efficient mobile solution.

The application is a mobile solution that addresses campus facility problems. Users may submit issue descriptions, while authorized workers manage task assignments, track progress, add update notes/photos, and confirm resolution to maintain regulated and organized workflows. Key functions include issue categorization, location-based reporting, worker assignment, status and priority updates, and history tracking.

## 1.2 Definitions and Acronyms

**1. Ticket:** A digital maintenance request submitted through the mobile application to report a facility-related issue.

**2. Reporter (Community Member):** A student who submits a maintenance ticket using the system.

**3. Facility Manager:** A user responsible for reviewing submitted tickets, assigning tasks to workers, setting ticket priorities, and monitoring issue resolution.

**4. Worker:** A maintenance staff member who receives assigned tasks, posts progress comments, and updates the status after resolving issues.

**5. Admin:** A system administrator responsible for managing user accounts, activating/deactivating accounts, and overall system control.

**6. RBAC (Role-Based Access Control):** A security mechanism that restricts system access based on predefined user roles.

**7. Dashboard:** An interface that provides an overview of maintenance tickets and their current status.

**8. Comment:** Text or photographic proof added to a ticket by workers or facility managers to document progress.

**9. Priority:** The urgency level of a ticket (Low, Medium, High), managed by Facility Managers.

**10. Status:** The current state of a ticket (Pending, Assigned, In Progress, Resolved, Denied).

**11. Workflow:** The sequence of steps from ticket submission to issue resolution.

# 2. Functional Requirements

## 2.1. User Stories

## Community Member

1. As a Community Member, I want to register and log in to the system, so that I can securely access my account.

2. As a Community Member, I want to log out, so that my account remains secure on shared devices.

3. As a Community Member, I want to delete my account, so that I can manage my personal data privacy.

4. As a Community Member, I want to select my role during registration, so that the system assigns the correct access level.

5. As a Community Member, I want my account to be approved before activation, so that only authorized users can access the system.

6. As a Community Member, I want to select a category and provide a description for the issue, so that it is properly classified and understood.

7. As a Community Member, I want to upload a photo of the issue from my gallery or camera, so that the maintenance team can visually understand the problem.

8. As a Community Member, I want to enter the building, floor, room number, and additional location notes when submitting a ticket, so that the assigned worker can easily locate the issue.

9. As a Community Member, I want to receive an on-screen confirmation message after submitting a ticket, so that I know my request has been successfully recorded.

10. As a Community Member, I want to view all my submitted tickets on a dashboard, so that I can check their status.

11. As a Community Member, I want to track the status of my submitted tickets (Pending, Assigned, In Progress, Resolved, Denied), so that I stay informed of their progress.

12. As a Community Member, I want to view comments and progress photos attached to my tickets, so that I can see exactly what work has been done.

## Worker

1. As a Worker, I want to register and select the worker role, so that I receive tasks relevant to my responsibilities.

2. As a Worker, I want to log in and log out securely.

3. As a Worker, I want to view my newly assigned tasks in my dashboard, so that I am aware of new responsibilities.

4. As a Worker, I want to view the details of my assigned tickets, including description, photo, location, and priority, so that I can perform the task properly.

5. As a Worker, I want to update the task status (e.g., In Progress, Resolved), so that the system reflects accurate progress.

6. As a Worker, I want to upload a completion photo to the ticket's comments, so that proof of work is documented.

7. As a Worker, I want to add text comments to the ticket, so that I can communicate progress or issues to the Facility Manager and Reporter.

## Facility Manager

1. As a Facility Manager, I want to log in and log out securely.

2. As a Facility Manager, I want to verify and activate or deactivate worker accounts, so that system security is maintained.

3. As a Facility Manager, I want to view all submitted tickets in a centralized management dashboard, so that I can monitor maintenance activities.

4. As a Facility Manager, I want to filter and sort tickets using status-based tabs, so that I can efficiently manage the ticket lifecycle.

5. As a Facility Manager, I want to assign pending tickets to specific workers, so that responsibilities are clearly defined.

6. As a Facility Manager, I want to set priority levels (Low, Medium, High) for tickets, so that critical issues are handled first.

7. As a Facility Manager, I want to update the ticket status, so that I can override or correct the current state if necessary.

## System Admin

1. As an Admin, I want to log in and log out securely.

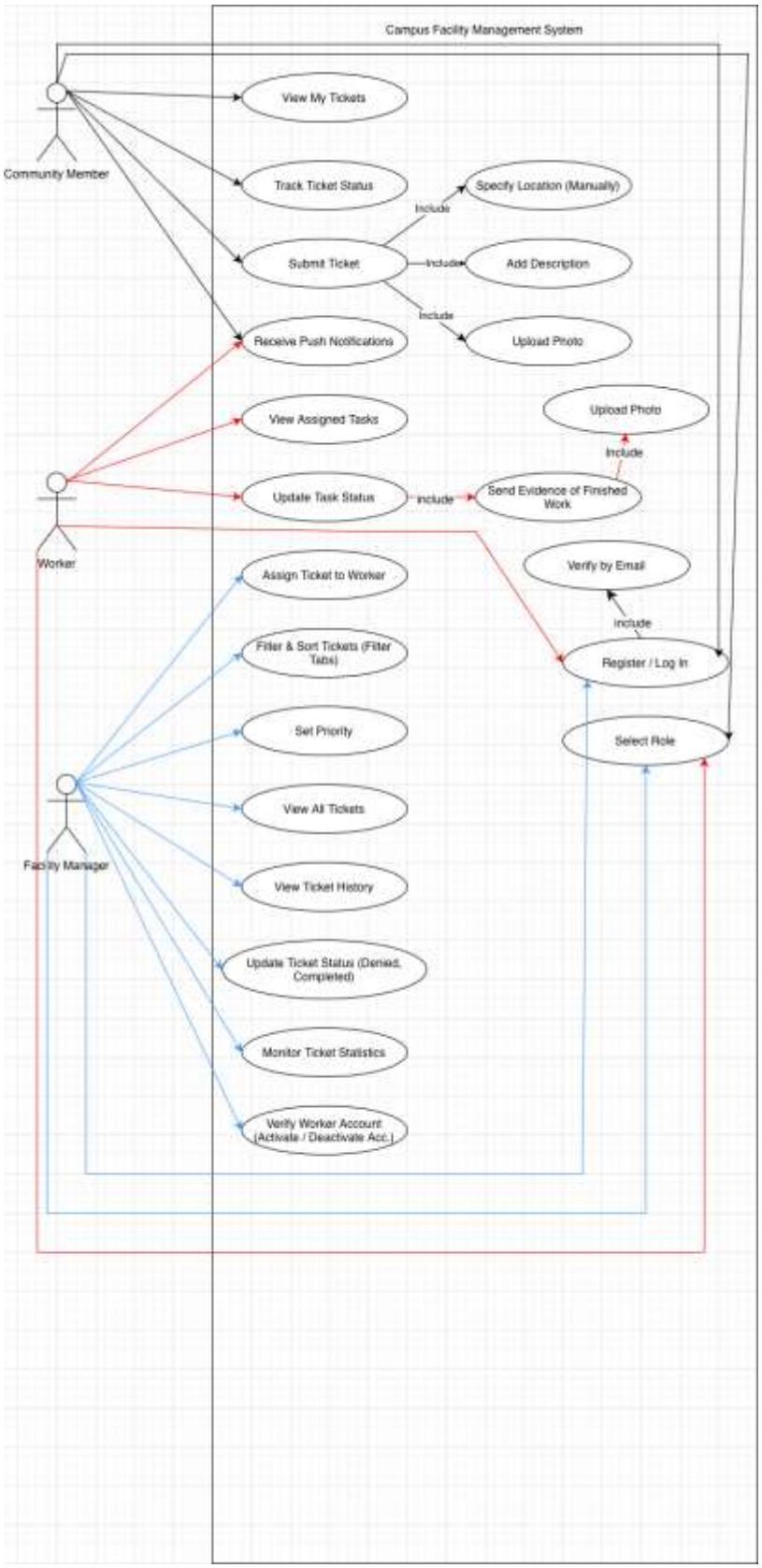
2. As an Admin, I want to view a list of all users in the system, so that I can manage the community.

3. As an Admin, I want to activate or deactivate user accounts, so that system access rules and security are enforced.

GERMAN INTERNATIONAL UNIVERSITY
الجامعة الألمانية الدولية

## 2.2 UML Use-Case Diagram

GERMAN INTERNATIONAL UNIVERSITY
الجامعة الألمانية الدولية



## 3. Non-functional Requirements

**Response Time:**

The system must ensure that all page load times and API responses are completed in under three 3 seconds under normal operating conditions. This includes loading dashboards, viewing issue details, updating statuses, and retrieving assigned tasks. Issue submission, including image upload, description entry, and location selection, should be processed efficiently and completed within a few seconds on a stable internet connection. Maintaining fast response times is critical to ensuring a smooth and frustration-free user experience, encouraging community members and staff to use the application regularly without delays.

**Concurrent Users:**

The application must support at least 500 simultaneous users without noticeable performance degradation. This includes users submitting issues, facility managers assigning tasks, and workers updating statuses at the same time. The backend system should be designed to efficiently manage multiple concurrent API requests, database transactions, and image uploads. Proper server configuration, load management, and scalable architecture must be implemented to ensure system stability during peak academic hours or high-demand periods.

**API Security:**

The system must implement strong API security measures to protect against unauthorized access and malicious attacks. Rate limiting must be applied to prevent abuse, spam submissions, or denial-of-service attempts. All input data must be validated and sanitized before processing to prevent SQL injection, cross-site scripting (XSS), and other common vulnerabilities. Authentication and role-based authorization must be enforced to ensure that users can only access features and data permitted by their role (e.g., community member, worker, facility manager, admin). All communication between the mobile application and backend must be encrypted using HTTPS.

**Backup & Recovery:**

Automated daily backups of the PostgreSQL database must be performed to protect all stored data, including user accounts, issue records, and status updates. Backup files should be securely stored and periodically tested to ensure successful restoration when needed. A disaster recovery plan must be documented and implemented to restore the system within an acceptable timeframe in case of hardware failure, cyberattack, or data corruption. These measures ensure business continuity and minimize data loss risks.

**Usability:**

The mobile application must provide an intuitive, simple, and user-friendly interface that allows community members to submit tickets in under one minute. The issue reporting process should require minimal steps: capturing or uploading a photo, writing a short description, and selecting the location. Navigation should be clear, buttons should be properly labeled, and status updates should be easy to understand (e.g., Submitted, Assigned, In Progress, Resolved). The design should follow consistent layout patterns to reduce confusion and ensure that users of different technical skill levels can comfortably use the system.

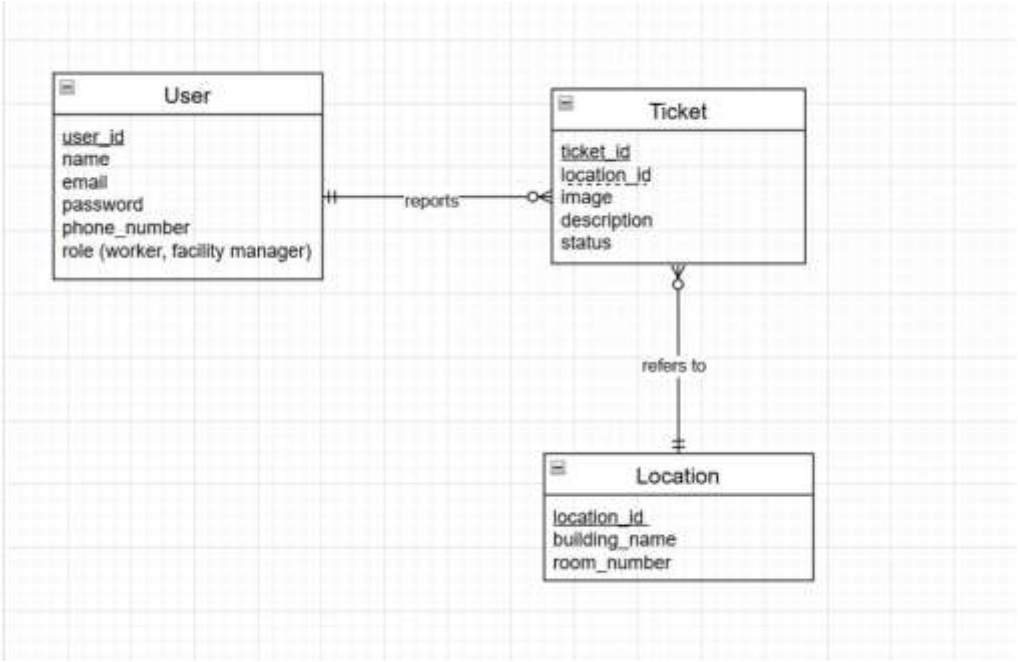
**Reliability:**

The system must operate consistently without crashes, errors, or data loss. It must handle image uploads robustly, even when network connections are unstable, by providing retry mechanisms and clear error messages if a submission fails. All issue updates and status changes must be accurately stored in the database and reflected immediately for authorized users. Logging and monitoring mechanisms should be implemented to detect system failures or unexpected behavior, ensuring that problems are quickly identified and resolved.

## 4. Constraints

• The system is initially designed for use within GIU campus only.

## 5. Entity Relation Diagram




## Technology Stack

• Response Time: Page load times under three seconds
• Concurrent Users: Support for at least 500 simultaneous users
• API Security: Rate limiting and input validation
• Backup & Recovery: Automated daily backups with disaster recovery plan